CS528 Homework 3

HTTP microservices and Pub/Sub

Yide Fang
Boston University | fangyd@bu.edu

Experiments: September 29-30, 2026 (UTC). All results below come from saved responses, screenshots, exported logs, and explicitly identified offline tests.

Python code: CS528-HWs / CS528-HW3

Service: https://cs528-hw3-files-sw2da6eioq-uc.a.run.app

Architecture and outcome

Component	Role
HTTP client / curl / browser	Request a file or supply a simulated X-country header.
Gen 2 cloud function	GET/POST file service; status handling; country filter; Pub/Sub publisher.
Pub/Sub topic + pull subscription	Deliver forbidden-request events independently of the HTTP response.
Mac subscriber	Print the error, append the shared bucket log, then acknowledge.
Cloud Storage	Existing data/ objects; one hw3-logs/forbidden-requests.jsonl object.

The file service returned original bytes for GET and POST. The baseline instructor client completed 100 successful GET requests. Country tests produced ten denied responses and one allowed response; all ten new events were later persisted without duplicate event IDs.

Platform limitation: CONNECT and TRACE returned 405 at Google Cloud's managed HTTP frontend before the Python function was invoked. Cloud Run does not forward these methods to its services [3]. Local application tests returned 501. The report records the cloud responses as a platform limitation relative to Step 3, not as successful public 501 cases.

Deployed resources

Setting	Value
Project / region	dazzling-seat-508219-j3 / us-central1
Function / runtime	cs528-hw3-files / Python 3.12 / GEN_2 / ACTIVE
Memory / timeout / maximum instances	256 MiB / 60 seconds / 2
Existing HW2 data	cs528-hw2-yf-2026; 12,000 HTML objects, data/0.html to data/11999.html
Same runtime identity for both services	cs528-hw3-sa@dazzling-seat-508219-j3.iam.gserviceaccount.com

Configure the project and messaging

Prerequisites are the existing HW2 project and bucket, Python 3.12, gcloud, curl, and the instructor-provided Mac arm64 client. The commands below document the configuration used; already-created resources do not need to be created again. The complete commands are also in the repository README.

Shared settings

```
export HW3_PROJECT="dazzling-seat-508219-j3"
export HW3_BUCKET="cs528-hw2-yf-2026"
export HW3_REGION="us-central1"
export HW3_USER="fangyd@bu.edu"
export HW3_SA="cs528-hw3-sa@${HW3_PROJECT}.iam.gserviceaccount.com"
export HW3_TOPIC="cs528-hw3-forbidden"
export HW3_SUB="cs528-hw3-forbidden-sub"
```

Authenticate the CLI and enable APIs

```
gcloud auth login "$HW3_USER"
gcloud config set project "$HW3_PROJECT"
gcloud services enable \
  cloudfunctions.googleapis.com run.googleapis.com \
  cloudbuild.googleapis.com artifactregistry.googleapis.com \
  pubsub.googleapis.com storage.googleapis.com \
  logging.googleapis.com iam.googleapis.com \
  iamcredentials.googleapis.com --project="$HW3_PROJECT"
```

This is the normal gcloud CLI login. The local Python service uses explicit service-account impersonation, described in the authentication section. The prohibited application-default login workflow is not used.

Create the dedicated identity, topic and subscription

```
gcloud iam service-accounts create cs528-hw3-sa \
  --display-name="CS528 HW3 Service Account" --project="$HW3_PROJECT"
gcloud pubsub topics create "$HW3_TOPIC" --project="$HW3_PROJECT"
gcloud pubsub subscriptions create "$HW3_SUB" \
  --topic="$HW3_TOPIC" --ack-deadline=60 --project="$HW3_PROJECT"
gcloud storage objects describe "gs://${HW3_BUCKET}/data/0.html"
```

The pull subscription uses a 60-second acknowledgement deadline. The subscriber runs on the Mac, so a laptop pause or process exit does not change the cloud function deployment; unacknowledged events can accumulate in the subscription.

Grant resource-scoped permissions

Use the shared settings above. The service account reads the HW2 dataset and publishes messages. The same account consumes messages on the Mac and updates only the designated log prefix through the conditional write grant.

```
gcloud storage buckets add-iam-policy-binding "gs://${HW3_BUCKET}" \
  --member="serviceAccount:${HW3_SA}" \
  --role=roles/storage.objectViewer --condition=None

HW3_LOG_EXPR="resource.name.startsWith('projects/_/buckets/${HW3_BUCKET}/objects/hw3-logs/')"
gcloud storage buckets add-iam-policy-binding "gs://${HW3_BUCKET}" \
  --member="serviceAccount:${HW3_SA}" --role=roles/storage.objectUser \
  --condition="title=hw3-log-write,expression=${HW3_LOG_EXPR}"

gcloud pubsub topics add-iam-policy-binding "$HW3_TOPIC" \
  --member="serviceAccount:${HW3_SA}" --role=roles/pubsub.publisher \
  --project="$HW3_PROJECT"
gcloud pubsub subscriptions add-iam-policy-binding "$HW3_SUB" \
  --member="serviceAccount:${HW3_SA}" --role=roles/pubsub.subscriber \
  --project="$HW3_PROJECT"

gcloud iam service-accounts add-iam-policy-binding "$HW3_SA" \
  --member="user:${HW3_USER}" --role=roles/iam.serviceAccountUser \
  --project="$HW3_PROJECT"
gcloud iam service-accounts add-iam-policy-binding "$HW3_SA" \
  --member="user:${HW3_USER}" --role=roles/iam.serviceAccountTokenCreator \
  --project="$HW3_PROJECT"
```

Why these roles are different

storage.objectViewer provides reads. The conditional **storage.objectUser** grant allows the repeated create/read/update/delete operations needed for replacing the log object, restricted to hw3-logs/. **pubsub.publisher** is attached to the topic and **pubsub.subscriber** to the subscription.

The human account receives **iam.serviceAccountUser** to attach the identity during deployment and **iam.serviceAccountTokenCreator** to mint its short-lived credentials. These grants serve different purposes [1].

Verify impersonation

```
gcloud storage objects describe "gs://${HW3_BUCKET}/data/0.html" \
  --account="$HW3_USER" --impersonate-service-account="$HW3_SA" \
  --project="$HW3_PROJECT"
```

An initial impersonation check was denied; a later object read using explicit user and service-account flags succeeded. The successful response identified data/0.html as a 97,586-byte text/html object. No service-account key file was required.

Deploy the function and run the Mac service

Cloud deployment

```
# From CS528-HW3; run the settings block in this terminal first.
cd cloud_function
gcloud functions deploy cs528-hw3-files \
  --gen2 --runtime=python312 --region="$HW3_REGION" \
  --project="$HW3_PROJECT" --source=. --entry-point=get_file \
  --trigger-http --allow-unauthenticated --service-account="$HW3_SA" \
  --set-env-vars="BUCKET_NAME=${HW3_BUCKET},PROJECT_ID=${HW3_PROJECT},TOPIC_ID=${HW3_TOPIC}" \
  --memory=256Mi --timeout=60s --min-instances=0 --max-instances=2
cd ..
```

main.py and requirements.txt are deployed from cloud_function/. The get_file handler gets runtime credentials from the attached service account. The exported function description confirms ACTIVE, GEN_2, the configured environment variables, and the service-account email listed with the deployed resources. The Cloud Build identity is a separate deployment detail, not the application runtime identity.

Local subscriber in terminal A

```
# From CS528-HW3 in terminal A; leave this process running.
python3 -m venv local_service/.venv
local_service/.venv/bin/python -m pip install -r local_service/requirements.txt
mkdir -p results
local_service/.venv/bin/python -u local_service/subscriber.py \
  2>&1 | tee -a results/subscriber-stage3.txt

Authenticated as: cs528-hw3-sa@dazzling-seat-508219-j3.iam.gserviceaccount.com
Listening on: projects/dazzling-seat-508219-j3/subscriptions/cs528-hw3-forbidden-sub
Log object: gs://cs528-hw2-yf-2026/hw3-logs/forbidden-requests.jsonl
```

Keep terminal A running while sending requests from terminal B. The subscriber processes one message at a time. After printing FORBIDDEN, it persists the JSON event, prints APPENDED (or ALREADY_STORED for a repeat), and acknowledges the message. A failed write leaves the message available for retry.

Cloud function request handling

For GET, the filename comes from the path; for POST, it comes from the JSON payload field filename. The bucket prefix is optional. Only data/ is served. For requests reaching the Python handler, missing files produce 404, other methods 501, invalid POST payloads 400, and storage/authentication failures 500. Each application error produces a simple print line and a structured JSON line on stdout [5]. CONNECT/TRACE are rejected earlier by the managed platform, as explained below.

HTTP demonstrations and the 100-request client

curl demonstrations

```
# Terminal B, from CS528-HW3.
HW3_URL="https://cs528-hw3-files-sw2da6eioq-uc.a.run.app"
curl -sS -o results/get-200-body.html -w '%{http_code}\n' \
  "$HW3_URL/data/0.html"
curl -sS -X POST -H 'Content-Type: application/json' \
  --data '{"filename":"data/0.html"}' \
  -o results/post-200-body.html -w '%{http_code}\n' "$HW3_URL/"
curl -sS -o results/get-404-body.txt -w '%{http_code}\n' \
  "$HW3_URL/data/not-a-real-hw3-file.html"
curl --http1.1 -sS -X PUT --data-binary '' \
  -o results/put-501-body.txt -w '%{http_code}\n' "$HW3_URL/data/0.html"
```

Case	Observed result
GET data/0.html	200; original bytes verified
POST JSON filename=data/0.html	200; original bytes verified
GET a nonexistent object	404
PUT with an explicit empty body	501; HTTP method not implemented.

GET and POST bodies both matched the HW2 object MD5, base64: **41U1Zt3txmd/WLuF7KDD5g==**. An HTTP/1.1 PUT without Content-Length initially received frontend 411; adding --data-binary " corrected the request and produced 501. The Cloud Logging export records the corrected PUT as status 501 at 2026-09-30T03:52:51.865994Z.

Evidence note: put-501-headers.txt and put-501-body.txt retain an earlier PUT 501 response. The separately named put-501-with-length files were later reused for a TRACE 405 response and are not used as PUT evidence. The original 411 result remains in the HTTP-method test summary.

Instructor-provided Mac client

```
chmod u+x /Users/thomasfang/Downloads/http-client
/Users/thomasfang/Downloads/http-client \
  -d cs528-hw3-files-sw2da6eioq-uc.a.run.app -p 443 -s \
  -b /cs528-hw2-yf-2026 -w data -n 100 -i 11999 -r 0 -t 60 -v \
  > results/client-100.txt 2>&1
HW3_CLIENT_EXIT=$?
printf '%s\n' "$HW3_CLIENT_EXIT" > results/client-100-exit.txt
```

```
{
  "client_exit_code": 0,
  "requests": 100,
  "http_status_counts": {
    "200": 100
  },
  "passed": true
}
```

This 100-request run occurred before the country filter was added. It proves the baseline GET service, not that the final filtered service returns 200 for every country. The client emits X-country; a local capture observed Cabo Verde. The slash in -b makes its HTTP request path begin with / and does not change the bucket name. Output is saved in results/client-100.txt.

Browser evidence and HTTP-method limits

The browser was opened on the service origin. Same-origin fetch calls exercised GET, POST, a missing file, and PUT; the console table displays the observed statuses.

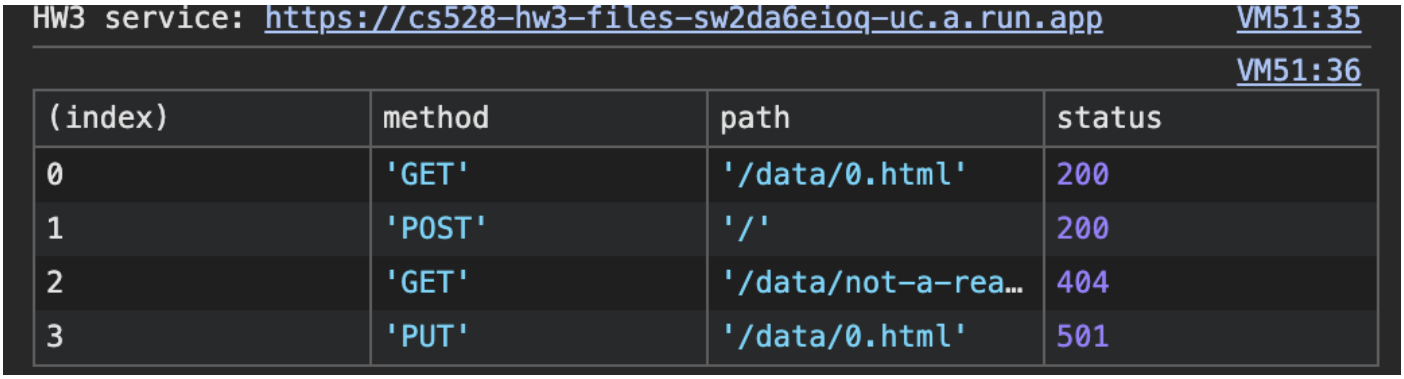


Figure 1. Actual browser console: GET 200, POST 200, missing file 404, and PUT 501.



Figure 2. Actual browser error entries for the intentional 404 and 501 requests.

Method	Public endpoint	Application-only test
PUT	501 after adding Content-Length: 0	501 + both log formats
DELETE / HEAD / OPTIONS / PATCH	501 for each method	501 + both log formats
CONNECT / TRACE	405 from the Google frontend	501 + both log formats

Why CONNECT and TRACE returned 405

Both public-endpoint captures contain HTTP/1.1 405 Method Not Allowed and a Google-generated HTML error page. Google documents that Cloud Run does not forward CONNECT or TRACE to services [3]. Together, these observations and the documented restriction attribute the rejection to Google's managed HTTP frontend serving the Cloud Run endpoint, before the request reaches our Python function. The requests reach Google Cloud; they are not blocked on the laptop or before entering the cloud.

```
Public endpoint:
CONNECT / TRACE -> Google-managed HTTP frontend -> 405
                   Python function is not invoked

Local application test:
CONNECT / TRACE -> Python handler directly -> 501 + application logs
```

The platform generates the 405 response. Because the function is not invoked, its configured 501 branch and its print/structured logging statements cannot execute for these requests. Platform request logs, if available, are distinct from the two application log formats required by Step 3. The saved response captures are retained under results/http-final-20260930T034845759407Z/.

The implementation keeps the generic 501 response for every non-GET/POST method that reaches the handler. No special CONNECT/TRACE workaround or VM gateway is added. The local Functions Framework tests verify this handler behavior; they do not prove a public 501 response. These two cloud cases are recorded as documented platform limitations relative to Step 3, and the saved test results remain unchanged.

Country filtering and persisted notifications

For GET/POST, X-country is normalized and checked before reading a file. The assignment list is North Korea, Iran, Cuba, Myanmar, Iraq, Libya, Sudan, Zimbabwe, and Syria. Common formal-name aliases are supported. An event is published and its publication future is awaited before returning 400. If publication cannot be confirmed, the response is 503 and no file is served.

```
curl -sS -H 'X-country: Iran' \
  -o results/iran-400-body.txt -w '%{http_code}\n' "$HW3_URL/data/0.html"
# Run once to send the complete batch and verify new persisted events:
python3 scripts/verify_country_flow.py
# To recheck the recorded original batch without sending more requests:
python3 scripts/verify_country_flow.py \
  --recheck results/country-tests-20260930T031158524468Z
```

Request group	HTTP outcome	New events saved
One GET from each of the nine specified countries	9 x 400	9
POST from Cuba	400	1
GET from Cabo Verde	200; original bytes match	0
Complete batch	11 of 11 HTTP checks passed	10 unique new events

One actual persisted event


```
{
  "event_id": "cd855670-b2d1-4e64-80fa-c3e471527eb6",
  "timestamp": "2026-09-30T03:12:08.387061+00:00",
  "event": "forbidden_country",
  "status": 400,
  "country": "Cuba",
  "country_header": "Cuba",
  "method": "POST",
  "path": "/",
  "message": "Permission denied: request from Cuba.",
  "pubsub_message_id": "21290348005742704",
  "received_at": "2026-09-30T03:38:49.618755+00:00"
}
```


The single destination object is **hw3-logs/forbidden-requests.jsonl** in the existing HW2 bucket. The final snapshot contains 11 unique event IDs: the initial Iran demonstration plus ten events from this batch. Every prior event remains present, and the new events match the expected country, method and path combinations. The successful recheck summary and subscriber stdout are retained in results/.

Cloud Logging evidence

The exported results/cloud-errors-final.json contains 44 application log records. The following counts are observed records across the demonstrations, not per-request guarantees inferred from source code. GCP extracts the severity field into the log envelope; the remaining structured fields are in jsonPayload.

HTTP status	Structured records	Plain print records
400	11	11
404	3	3
501	8	8

404: actual print line and structured payload

Structured record timestamp: 2026-09-30T03:48:49.255034Z

```
404 file_not_found: File not found.
{
  "country": "Cabo Verde",
  "event": "file_not_found",
  "message": "File not found.",
  "method": "GET",
  "object_name": "data/not-a-real-hw3-file-20260930T034845759407Z.html",
  "path": "/data/not-a-real-hw3-file-20260930T034845759407Z.html",
  "status": 404
}
```

501: actual print line and structured payload

Structured record timestamp: 2026-09-30T03:52:51.865994Z

```
501 unsupported_method: HTTP method not implemented.
{
  "country": "Cabo Verde",
  "event": "unsupported_method",
  "message": "HTTP method not implemented.",
  "method": "PUT",
  "object_name": null,
  "path": "/data/0.html",
  "status": 501
}
```

400: actual print line and structured payload

Structured record timestamp: 2026-09-30T03:12:08.441530Z

```
400 forbidden_country: Permission denied: request from Cuba.
{
  "country": "Cuba",
  "country_canonical": "Cuba",
  "event": "forbidden_country",
  "event_id": "cd855670-b2d1-4e64-80fa-c3e471527eb6",
  "message": "Permission denied: request from Cuba.",
  "method": "POST",
  "object_name": null,
  "path": "/",
  "pubsub_message_id": "21290348005742704",
  "status": 400
}
```

Payloads are reproduced above; verbose instance labels and other GCP envelope fields are omitted here and remain in the JSON export. The latest 501 example is the corrected PUT request. CONNECT/TRACE frontend responses are documented in the HTTP-method limits section.

Authentication, recovery and interpretation

Keyless impersonation on the laptop

The chosen mechanism is service-account impersonation, with no downloaded private key and no application-default login. The existing gcloud login identifies fangyd@bu.edu as the principal allowed to mint credentials. Token Creator on the target service account authorizes a short-lived access token [1]. Both Python API clients are explicitly given a GcloudImpersonatedCredentials object, so their data requests use the same service account as the cloud function.

The adapter calls the installed Cloud SDK config config-helper with --force-auth-refresh, --account and --impersonate-service-account. It captures the token and its actual expiration in memory, and google-auth invokes refresh as needed. Tokens are not printed or saved by the program. This helper is a hidden SDK command, verified against the installed version; the adapter may require maintenance after an SDK change.

This choice avoids creating and distributing a long-lived service-account key while retaining explicit control over the identity and its resource permissions. The function description and local startup output confirm the intended shared account. A normal gcloud user login is used only as the starting identity for impersonation.

One-file append, duplicate handling and restart recovery

For this small log, the subscriber reads the current object and writes the previous contents plus a JSON line. Generation preconditions prevent an unnoticed concurrent overwrite; generation 0 is used for initial creation [2]. Persisted event IDs let a restarted subscriber recognize an already-written event. The message is acknowledged only after the event is durably present. This implements deduplication, not exactly-once Pub/Sub delivery.

The original subscriber exited after a 503 connection reset. Thus the first batch check found zero new stored events despite all HTTP checks passing. After adding bounded exponential backoff and manually restarting the service, it consumed and acknowledged all ten queued events. The recheck passed without resending the HTTP requests. Automatic retry after an injected connection reset passed offline tests; the live evidence demonstrates recovery following the manual restart [4].

Limits and test scope

X-country is an untrusted client-supplied simulation, not actual geolocation. The HW2 bucket retained public object-read access, so direct public URLs bypass the function, and log objects inherit that public read access. The function itself serves only data/. Object replacement and full-log scans are suitable for this small assignment, not a high-volume production log.

Offline checks passed: 17 HTTP-handler tests, 18 subscriber/authentication/storage tests, and 4 recheck tests. Those checks simulate Google APIs and are distinguished from the observed cloud results. CONNECT/TRACE retain their observed platform-generated 405 responses in the public-endpoint evidence; they are not relabeled as successful 501 tests.

References

[1] Service account impersonation

[2] Cloud Storage request preconditions

[3] Cloud Run known issues: HTTP methods

[4] Pub/Sub error codes and retry guidance

[5] Structured logging